

Aula de Banco de Dados: DDL e DML

Versão do Aluno — com exemplos guiados e roteiro prático sem gabarito

1. Identificação da aula

Item	Descrição
Tema	Banco de Dados: comandos DDL e DML
Público-alvo	Alunos iniciantes do curso Técnico em Desenvolvimento de Sistemas
Duração sugerida	2 aulas de 50 minutos ou 1 encontro estendido de 100 minutos
Ferramenta sugerida	MySQL, MySQL Workbench, phpMyAdmin, DB Fiddle ou OneCompiler SQL/MySQL
Organização	Explicação teórica, exemplos guiados, exercícios resolvidos e roteiro prático
Versão	Aluno

2. Objetivos de aprendizagem

- Compreender o que é SQL e por que ele é usado em bancos de dados relacionais.
- Diferenciar comandos DDL, que modificam a estrutura do banco, de comandos DML, que manipulam os dados.
- Criar tabelas simples usando CREATE TABLE, tipos de dados e restrições básicas.
- Inserir, consultar, atualizar e excluir registros usando INSERT, SELECT, UPDATE e DELETE.
- Alterar tabelas existentes usando ALTER TABLE com cuidado e planejamento.
- Criar e remover índices simples, entendendo seu impacto em consultas e operações de escrita.
- Testar comandos SQL e interpretar os resultados de forma segura.

3. Recursos necessários

- Computador com acesso à internet ou laboratório de informática.
- Ambiente para executar SQL/MySQL.
- Caderno para anotações e rascunho da lógica dos comandos.
- Projetor ou tela para acompanhamento dos exemplos guiados.
- Arquivo do roteiro prático ou este material impresso/digital.

4. Conteúdos essenciais

4.1 O que é SQL?

SQL significa Structured Query Language, ou Linguagem de Consulta Estruturada. Ela é usada para criar estruturas em bancos de dados, inserir informações, consultar registros, atualizar dados e remover dados quando necessário.

Em uma loja virtual, por exemplo, o SQL pode ser usado para criar uma tabela de produtos, cadastrar novos produtos, buscar produtos por categoria, alterar preços e remover registros incorretos.

4.2 Diferença entre DDL e DML

Grupo	Significado	Função principal	Exemplos
DDL	Data Definition Language	Define ou altera a estrutura do banco de dados.	CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX, DROP INDEX
DML	Data Manipulation Language	Manipula os dados armazenados nas tabelas.	INSERT, SELECT, UPDATE, DELETE

Observação: Uma forma simples de explicar: DDL mexe na estrutura, como criar ou alterar uma tabela. DML mexe nos dados dentro da estrutura, como cadastrar, consultar, atualizar ou excluir registros.

4.3 Analogia para iniciantes

Imagine uma escola com armários para guardar documentos. O DDL seria a ação de construir os armários, definir quantas gavetas existem e colocar etiquetas em cada gaveta. O DML seria a ação de guardar, consultar, trocar ou retirar documentos de dentro desses armários.

Situação	Em banco de dados
Construir uma gaveta para guardar fichas de clientes	CREATE TABLE clientes
Adicionar uma nova coluna chamada telefone	ALTER TABLE clientes ADD telefone VARCHAR(15)
Guardar uma ficha de cliente	INSERT INTO clientes ...
Procurar um cliente cadastrado	SELECT ... FROM clientes
Corrigir o e-mail de um cliente	UPDATE clientes SET email = ...
Remover um cadastro incorreto	DELETE FROM clientes WHERE ...

4.4 Comandos principais

Comando	Grupo	Para que serve
CREATE DATABASE	DDL	Cria um banco de dados.
CREATE TABLE	DDL	Cria uma tabela.
ALTER TABLE	DDL	Altera a estrutura de uma tabela existente.
DROP TABLE	DDL	Remove uma tabela.
CREATE INDEX	DDL	Cria um índice para melhorar buscas.
DROP INDEX	DDL	Remove um índice.
INSERT	DML	Insere registros.
SELECT	DML	Consulta registros.
UPDATE	DML	Atualiza registros existentes.
DELETE	DML	Remove registros.

4.5 Tipos de dados básicos

Tipo	Uso comum	Exemplo
INT	Números inteiros	id_cliente INT
VARCHAR(n)	Textos curtos com limite de caracteres	nome VARCHAR(100)
DATE	Datas sem horário	data_cadastro DATE
DATETIME	Data e horário	data_pedido DATETIME
DECIMAL(10,2)	Valores monetários ou decimais	preco DECIMAL(10,2)

4.6 Restrições importantes

Restrição	Significado
PRIMARY KEY	Identifica cada registro de forma única.
FOREIGN KEY	Relaciona uma tabela com outra.
NOT NULL	Impede que o campo fique vazio.
UNIQUE	Impede valores duplicados em uma coluna.
AUTO_INCREMENT	Gera números automaticamente, geralmente em chaves primárias.
CHECK	Define uma regra que o valor precisa obedecer.

5. Roteiro introdutório da aula

Nesta aula, você vai aprender a diferença entre criar a estrutura de um banco de dados e manipular os dados que ficam dentro dele. Para facilitar, pense em uma loja virtual: antes de cadastrar produtos, é preciso criar uma tabela para guardar nome, preço, categoria e estoque. Depois disso, é possível inserir, consultar, alterar e excluir produtos.

A parte que cria ou altera a estrutura recebe o nome de DDL. A parte que trabalha com os dados recebe o nome de DML.

6. Plano de aula

Momento	Tempo	Descrição
Acolhimento e contextualização	10 min	Apresentar a situação-problema de uma loja/empresa que precisa organizar dados.
Explicação teórica	20 min	Explicar SQL, DDL, DML, tabelas, tipos de dados, chaves e restrições.
Exemplos guiados	30 min	Executar comandos de criação, inserção, consulta, atualização, exclusão e alteração de tabela.
Exercícios resolvidos	25 min	Os alunos digitam, testam e observam o resultado dos comandos.
Roteiro prático	30 min	Atividade TechFix: modificar estruturas de tabelas e refletir sobre cuidados.
Fechamento e revisão	5 min	Retomar diferença entre DDL e DML e principais comandos.

7. Exemplos guiados em SQL/MySQL

Os exemplos a seguir podem ser digitados e testados em um ambiente MySQL. Eles usam uma loja fictícia chamada LojaTech.

7.1 Criando e selecionando o banco de dados

```
DROP DATABASE IF EXISTS loja_tech;
CREATE DATABASE loja_tech;
USE loja_tech;
```

O comando CREATE DATABASE cria o banco de dados. O comando USE informa qual banco será usado nos próximos comandos.

7.2 Criando uma tabela com CREATE TABLE

```
CREATE TABLE produtos (
  id_produto INT AUTO_INCREMENT PRIMARY KEY,
  nome_produto VARCHAR(100) NOT NULL,
  categoria VARCHAR(50),
  preco DECIMAL(10,2) NOT NULL,
  quantidade_estoque INT DEFAULT 0 CHECK (quantidade_estoque >= 0)
);
```

Esse comando cria a tabela produtos. A coluna id_produto identifica cada produto. A coluna nome_produto não pode ficar vazia. O preço usa DECIMAL, indicado para valores monetários.

7.3 Inserindo dados com INSERT

```
INSERT INTO produtos (nome_produto, categoria, preco, quantidade_estoque)
VALUES
('Mouse Gamer', 'Periféricos', 89.90, 20),
('Teclado Mecânico', 'Periféricos', 249.90, 10),
('Monitor 24 polegadas', 'Monitores', 899.90, 5);
```

O INSERT adiciona novos registros na tabela. Como id_produto é AUTO_INCREMENT, o banco gera o código automaticamente.

7.4 Consultando dados com SELECT

```
SELECT * FROM produtos;
```

```
SELECT nome_produto, preco
FROM produtos;
```

```
SELECT * FROM produtos
WHERE categoria = 'Periféricos';
```

O SELECT consulta dados. O asterisco (*) mostra todas as colunas. O WHERE filtra registros por uma condição.

7.5 Atualizando registros com UPDATE

```
UPDATE produtos
SET preco = 219.90
WHERE id_produto = 2;
```

```
SELECT * FROM produtos;
```

Observação: Nunca use UPDATE sem WHERE em atividades reais, porque isso pode alterar todos os registros da tabela.

7.6 Excluindo registros com DELETE

```
DELETE FROM produtos
WHERE id_produto = 3;
```

```
SELECT * FROM produtos;
```

Observação: Assim como no UPDATE, o DELETE sem WHERE pode apagar todos os registros da tabela.

7.7 Alterando a tabela com ALTER TABLE

```
ALTER TABLE produtos
ADD COLUMN marca VARCHAR(50);
```

```
ALTER TABLE produtos
MODIFY COLUMN nome_produto VARCHAR(150);
```

```
ALTER TABLE produtos
DROP COLUMN marca;
```

O ALTER TABLE modifica a estrutura da tabela. Ele pode adicionar colunas, alterar tipos/tamanhos e remover colunas.

7.8 Criando duas tabelas relacionadas

```
CREATE TABLE clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome_cliente VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);
```

```
CREATE TABLE pedidos (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    data_pedido DATETIME,
    valor_total DECIMAL(10,2),
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

A tabela pedidos possui id_cliente como chave estrangeira. Isso cria uma relação entre o pedido e o cliente que fez a compra.

7.9 Criando e removendo índices

```
CREATE INDEX idx_nome_produto
ON produtos (nome_produto);
```

```
DROP INDEX idx_nome_produto
ON produtos;
```

Índices podem melhorar a velocidade de consultas, mas em excesso podem prejudicar inserções, atualizações e exclusões, pois o banco precisa manter os índices atualizados.

8. Dez exercícios guiados com respostas para digitar e testar

Os exercícios desta seção possuem respostas porque fazem parte da prática guiada da aula teórica. A proposta é que o aluno digite, execute, observe o resultado e entenda o funcionamento dos comandos.

Exercício 1 — Criar uma tabela simples

O que o aluno deve praticar: Praticar CREATE TABLE e tipos de dados básicos.

```
CREATE TABLE categorias (
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,
    nome_categoria VARCHAR(80) NOT NULL
);
```

Explicação: A tabela categorias foi criada com uma chave primária automática e uma coluna obrigatória para o nome.

Exercício 2 — Inserir registros

O que o aluno deve praticar: Praticar INSERT INTO.

```
INSERT INTO categorias (nome_categoria)
VALUES
('Informática'),
('Periféricos'),
('Monitores');
```

Explicação: O comando adiciona três categorias. O id é gerado automaticamente pelo AUTO_INCREMENT.

Exercício 3 — Consultar todos os dados

O que o aluno deve praticar: Praticar SELECT com asterisco.

```
SELECT * FROM categorias;
```

Explicação: O SELECT * mostra todas as colunas e todos os registros da tabela categorias.

Exercício 4 — Consultar dados específicos com WHERE

O que o aluno deve praticar: Praticar filtro com WHERE.

```
SELECT * FROM categorias
WHERE nome_categoria = 'Periféricos';
```

Explicação: A consulta mostra apenas os registros que atendem à condição definida no WHERE.

Exercício 5 — Atualizar um registro

O que o aluno deve praticar: Praticar UPDATE com WHERE.

```
UPDATE categorias
SET nome_categoria = 'Acessórios'
WHERE id_categoria = 2;
```

```
SELECT * FROM categorias;
```

Explicação: O registro de id 2 é alterado. O SELECT final ajuda a conferir o resultado.

Exercício 6 — Remover um registro

O que o aluno deve praticar: Praticar DELETE com WHERE.

```
DELETE FROM categorias
WHERE id_categoria = 3;
```

```
SELECT * FROM categorias;
```

Explicação: O registro de id 3 é removido. O WHERE evita que todos os registros sejam apagados.

Exercício 7 — Criar tabela com chave primária

O que o aluno deve praticar: Praticar PRIMARY KEY e AUTO_INCREMENT.

```
CREATE TABLE fornecedores (
    id_fornecedor INT AUTO_INCREMENT PRIMARY KEY,
    nome_fornecedor VARCHAR(100) NOT NULL,
    telefone VARCHAR(15)
);
```

Explicação: A chave primária identifica cada fornecedor de forma única.

Exercício 8 — Criar duas tabelas com chave estrangeira

O que o aluno deve praticar: Praticar relacionamento entre tabelas.

```
CREATE TABLE departamentos (
    id_departamento INT AUTO_INCREMENT PRIMARY KEY,
    nome_departamento VARCHAR(100) NOT NULL
);

CREATE TABLE funcionarios (
    id_funcionario INT AUTO_INCREMENT PRIMARY KEY,
    nome_funcionario VARCHAR(100) NOT NULL,
    id_departamento INT,
    FOREIGN KEY (id_departamento) REFERENCES departamentos(id_departamento)
);
```

Explicação: A tabela funcionarios usa id_departamento como chave estrangeira para se relacionar com departamentos.

Exercício 9 — Alterar uma tabela com ALTER TABLE

O que o aluno deve praticar: Praticar adição de coluna.

```
ALTER TABLE fornecedores
ADD COLUMN email VARCHAR(100);
```

Explicação: A tabela fornecedores recebe uma nova coluna chamada email.

Exercício 10 — Criar e remover um índice

O que o aluno deve praticar: Praticar CREATE INDEX e DROP INDEX.

```
CREATE INDEX idx_nome_fornecedor
ON fornecedores (nome_fornecedor);

DROP INDEX idx_nome_fornecedor
ON fornecedores;
```

Explicação: O índice é criado para otimizar buscas pelo nome do fornecedor e depois removido.

9. Roteiro de atividade prática — Gerenciando dados da TechFix

Esta atividade foi adaptada para ficar mais clara e organizada. Ela trabalha principalmente alterações na estrutura das tabelas usando comandos DDL, especialmente ALTER TABLE.

Atenção: Versão do aluno: esta seção não contém gabarito nem comandos prontos de solução. Resolva as tarefas escrevendo seus próprios comandos SQL.

9.1 Contexto

A TechFix é uma empresa de tecnologia que está expandindo suas operações. A equipe percebeu que o banco de dados precisa ser ajustado para registrar novas informações, remover campos que não serão mais usados e alterar o tipo de dados de algumas colunas.

9.2 Banco de dados inicial

Execute os comandos abaixo para criar o banco de dados e as tabelas iniciais da atividade.

```
DROP DATABASE IF EXISTS techfix;
CREATE DATABASE techfix;
USE techfix;

CREATE TABLE clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL,
    cidade VARCHAR(50),
    data_nascimento DATE
);

CREATE TABLE produtos (
    id_produto INT AUTO_INCREMENT PRIMARY KEY,
    nome_produto VARCHAR(100) NOT NULL,
    preco DECIMAL(10,2) NOT NULL,
    fabricante VARCHAR(50),
    estoque INT
```

```
);

CREATE TABLE pedidos (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    data_pedido DATE,
    valor_total DECIMAL(10,2),
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
);
```

9.3 Dados de teste

Execute os dados abaixo para facilitar os testes após as alterações.

```
INSERT INTO clientes (nome, email, cidade, data_nascimento)
VALUES
('Ana Lima', 'ana@email.com', 'Barueri', '2005-03-10'),
('Carlos Souza', 'carlos@email.com', 'Osasco', '2004-07-22');

INSERT INTO produtos (nome_produto, preco, fabricante, estoque)
VALUES
('Mouse Óptico', 49.90, 'TechMouse', 30),
('Teclado USB', 79.90, 'KeyTech', 20);

INSERT INTO pedidos (id_cliente, data_pedido, valor_total)
VALUES
(1, '2026-05-20', 129.80),
(2, '2026-05-21', 79.90);
```

9.4 Desafio

Você deverá modificar a estrutura das tabelas da TechFix de acordo com as novas necessidades da empresa.

9.5 Tarefas para resolver

Tarefa 1: Adicionar uma coluna chamada `telefone_secundario` na tabela `clientes`, do tipo `VARCHAR(15)`.
Escreva aqui o comando ou sua resposta:

```
ALTER TABLE clientes
ADD telefone_secundario VARCHAR(15);
```

Tarefa 2: Remover a coluna `fabricante` da tabela `produtos`, pois ela não será mais utilizada.
Escreva aqui o comando ou sua resposta:

```
ALTER TABLE produtos
DROP COLUMN fabricante;
```

Tarefa 3: Alterar a coluna `data_pedido` da tabela `pedidos` de `DATE` para `DATETIME`, para armazenar data e horário.
Escreva aqui o comando ou sua resposta:

```
ALTER TABLE pedidos
MODIFY COLUMN data_pedido DATETIME;
```

Tarefa 4: Adicionar as colunas `peso DECIMAL(5,2)` e `dimensao VARCHAR(50)` na tabela `produtos`.
Escreva aqui o comando ou sua resposta:

```
ALTER TABLE produtos  
ADD peso DECIMAL(5,2),  
ADD dimensao VARCHAR(50);
```

Tarefa 5: Alterar o tamanho da coluna nome da tabela clientes de VARCHAR(50) para VARCHAR(100).
Escreva aqui o comando ou sua resposta:

```
ALTER TABLE clientes  
MODIFY COLUMN nome VARCHAR(100) NOT NULL;
```

Tarefa 6: Criar um índice para facilitar consultas pelo nome do produto.
Escreva aqui o comando ou sua resposta:

```
CREATE INDEX idx_nome_produto  
ON produtos(nome_produto);
```

Tarefa 7: Consultar a estrutura das tabelas para conferir se as alterações foram aplicadas.
Escreva aqui o comando ou sua resposta:

```
DESCRIBE clientes;  
DESCRIBE produtos;  
DESCRIBE pedidos;
```

Tarefa 8: Escrever uma breve reflexão sobre os cuidados necessários ao modificar tabelas em bancos reais.

Escreva aqui o comando ou sua resposta:

Modificar tabelas em bancos de dados exige cuidado, pois alterações podem afetar sistemas que utilizam essas informações. Antes de qualquer mudança, é importante realizar backups, testar em ambiente de desenvolvimento e verificar possíveis impactos em consultas, relatórios e aplicações. Dessa forma, evita-se perda de dados e interrupções no funcionamento do sistema.

9.6 Perguntas de reflexão

Por que é importante usar o comando ALTER TABLE com cuidado em bancos de dados de produção?

Porque esse comando altera a estrutura do banco de dados e pode causar perda de informações, falhas em aplicações ou interrupção de serviços. Em ambientes de produção, qualquer alteração deve ser planejada, testada e acompanhada de backups para evitar problemas.

Como você garantiria a integridade dos dados ao modificar o tipo de uma coluna?

Primeiro eu verificaria se os dados atuais são compatíveis com o novo tipo. Depois faria um backup do banco de dados e realizaria testes em um ambiente de desenvolvimento. Somente após confirmar que não haverá perda ou corrupção de dados a alteração seria aplicada em produção.

Quais desafios podem surgir ao remover colunas que podem estar sendo usadas em outros sistemas ou relatórios?

A remoção pode quebrar consultas SQL, relatórios, dashboards e aplicações que dependem daquela informação. Também pode causar erros em integrações com outros sistemas. Por isso, é necessário identificar todos os locais onde a coluna é utilizada antes de removê-la.

9.7 Checklist de entrega

- Criei o banco de dados inicial da TechFix.
- Executei as alterações solicitadas nas tabelas.
- Conferi a estrutura das tabelas após as alterações.
- Respondi às perguntas de reflexão.
- Testei os comandos antes de finalizar a entrega.
- Salvei ou enviei os comandos no formato solicitado pelo professor.

9.8 Critérios simples de avaliação

Critério	Valor sugerido
Uso correto dos comandos ALTER TABLE	4,0 pontos
Criação e conferência correta das tabelas	2,0 pontos
Uso adequado de tipos de dados e índices	1,5 ponto
Respostas reflexivas sobre cuidados em bancos reais	1,5 ponto
Organização, clareza e entrega no formato solicitado	1,0 ponto

10. Principais erros dos alunos iniciantes

Erro comum	Como evitar
Esquecer ponto e vírgula no final do comando	Conferir se cada comando SQL termina com ;
Usar UPDATE sem WHERE	Sempre verificar qual registro será alterado antes de executar.
Usar DELETE sem WHERE	Executar SELECT antes para conferir o registro que será excluído.
Confundir DDL com DML	Perguntar: estou mexendo na estrutura ou nos dados?
Criar coluna com tipo inadequado	Escolher tipos conforme o dado: texto, número, data ou valor monetário.
Remover coluna sem analisar impacto	Verificar se a coluna é usada por relatórios, sistemas ou integrações.
Criar índices demais	Criar índices apenas em colunas usadas em consultas frequentes.

11. Dicas para evitar problemas em SQL

- Antes de alterar ou excluir dados, execute um SELECT para conferir os registros afetados.
- Em bancos reais, faça backup antes de alterações importantes.
- Teste comandos em ambiente de desenvolvimento antes de usar em produção.
- Use nomes claros para tabelas, colunas e índices.
- Evite apagar tabelas ou colunas sem verificar dependências.
- Lembre-se: DDL altera a estrutura; DML manipula os dados.

12. Resumo do que foi aprendido

- SQL é uma linguagem usada para trabalhar com bancos de dados relacionais.
- DDL cria, altera ou remove estruturas do banco de dados.

- DML insere, consulta, atualiza e remove registros dentro das tabelas.
- CREATE TABLE cria tabelas e define colunas, tipos e restrições.
- ALTER TABLE modifica a estrutura de uma tabela existente.
- INSERT, SELECT, UPDATE e DELETE são comandos de manipulação de dados.
- Chaves primárias identificam registros; chaves estrangeiras relacionam tabelas.
- Índices podem melhorar buscas, mas precisam ser usados com planejamento.

13. Perguntas rápidas de revisão

Pergunta: Qual comando DDL cria uma nova tabela?

Resposta para revisão teórica: CREATE TABLE.

Pergunta: Qual comando DML insere registros em uma tabela?

Resposta para revisão teórica: INSERT.

Pergunta: Qual comando DML consulta dados?

Resposta para revisão teórica: SELECT.

Pergunta: Qual comando DML atualiza registros?

Resposta para revisão teórica: UPDATE.

Pergunta: Qual comando DML remove registros?

Resposta para revisão teórica: DELETE.

Pergunta: Qual comando DDL altera uma tabela existente?

Resposta para revisão teórica: ALTER TABLE.

Pergunta: Qual é o risco de UPDATE sem WHERE?

Resposta para revisão teórica: Alterar todos os registros da tabela.

Pergunta: Qual é o risco de DELETE sem WHERE?

Resposta para revisão teórica: Apagar todos os registros da tabela.

Pergunta: Para que serve uma chave primária?

Resposta para revisão teórica: Identificar cada registro de forma única.

Pergunta: Para que serve uma chave estrangeira?

Resposta para revisão teórica: Relacionar uma tabela com outra.

14. Referências-base dos anexos utilizados

Material elaborado a partir dos anexos fornecidos sobre conceitos de DDL e DML, comandos DDL, criação de tabelas e índices, modificação de estruturas de dados e roteiro prático TechFix. As referências originais dos slides incluem materiais sobre SQL, modelagem de banco de dados, comandos DML em MySQL e fundamentos de sistemas de banco de dados.